

COUNT MASTER

Technical Documentation & Tuning Guide

Project: Count Master — hyper-casual crowd-runner
Engine: Unity 2022.3.62f3 (LTS) — Built-in Render Pipeline
Platform: Android (portrait)
Levels: Level1, Level2 (Assets/_Game/Scenes/)
Audience: Junior developers joining the project
Date: 29 July 2026

You steer a crowd of characters down a lane. Math gates add or multiply your count, spinning hurdles kill individual runners, enemy groups fight your crowd one-for-one, and if anyone is left alive at the finish line — you win.

0. How to read this document

This document explains the whole codebase for someone who has never opened this project. It is organised so you can read it front-to-back once, then use sections 5–7 as a reference while working.

1. Game overview	What the game is and the core loop in one diagram.
2. Managers & design patterns	The <i>why and how</i> of every pattern used: singletons, the state machine, object pooling, composition.
3. Script reference	Every script we wrote: what it does, its public API and who calls it.
4. Game flow, start to end	The full runtime walkthrough: scene load → first touch → gates/hurdles/battles → finish → next level.
5. Parameter reference & tuning guide	Every Inspector field of every component, its default, its current scene value, and how to tweak it.
6. Level layouts	Where every gate, enemy group and hurdle sits in Level1 and Level2, with the balance math.
7. Known issues & gotchas	Crash points, magic numbers, stale prefabs and performance notes. Read before touching anything.

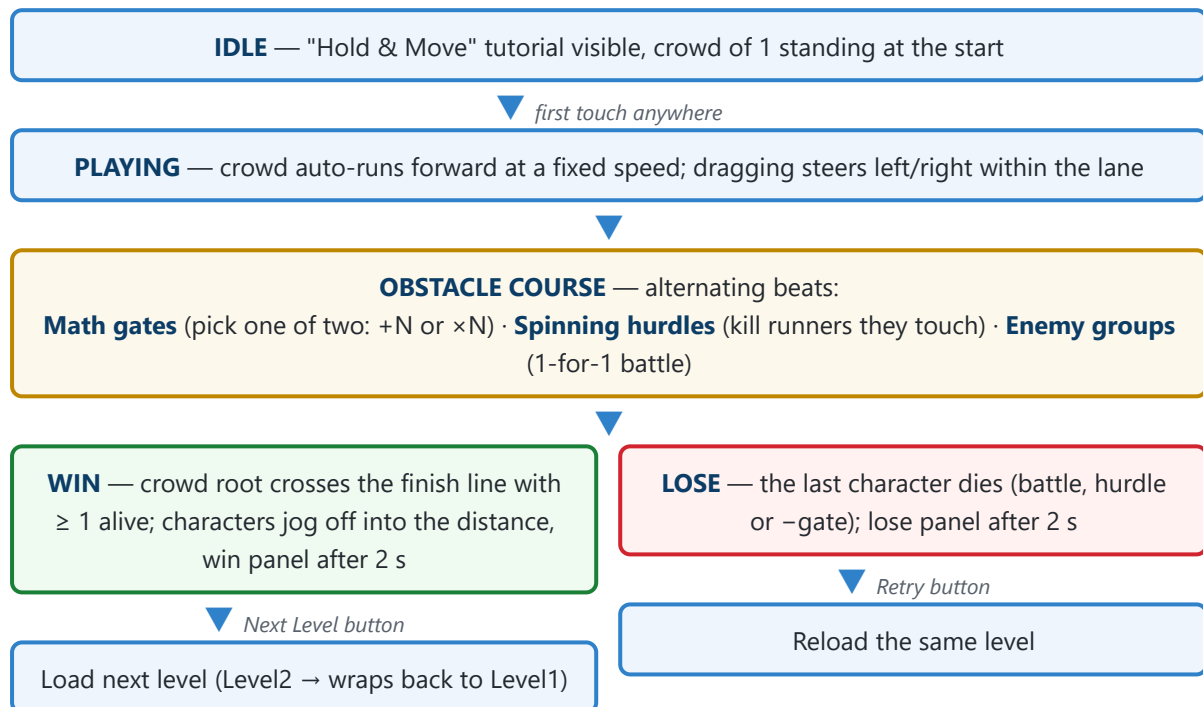
Conventions in this doc

Callout boxes mean: **red** = will bite you, **green** = practical tip, **amber** = the reasoning ("why"), **blue** = good-to-know. Field tables always show the **code default** (the value in the C# file) and the **scene value** (what is actually serialized in the level) separately — in Unity these can differ, and in this project they sometimes do.

1. Game overview

Count Master is a hyper-casual crowd-runner. The player holds and drags to steer a crowd that auto-runs forward. The goal of every level is simple: keep at least one character alive until the finish line.

The core loop



Project facts

- Two scenes ship in Build Settings: `Assets/_Game/Scenes/Level1.unity` and `Level2.unity`. (`Assets/Scenes/SampleScene.unity` is the leftover Unity default — ignore it.)
- All gameplay code lives in `Assets/_Game/Scripts/` — 21 scripts, grouped by role: `Managers/` `Player/` `Enemy/` `Level/` `Boss/` `Pool/` `UI/`.
- No namespaces, no assembly definitions, no base classes.** Everything is a plain `MonoBehaviour` compiled into `Assembly-CSharp`. This is deliberate — see section 2.4.
- Player and enemy characters are **the same rigged model**; only the material differs (`Player_Blue.mat` vs `Enemy_Red.mat`).
- All UI text is TextMeshPro. The gate labels use the font `Assets/_Ripped/Font/LilitaOne_1 SDF.asset`.

2. Managers & design patterns — the why and how

2.1 The Singleton pattern (7 managers)

Seven classes are **singletons**: exactly one instance exists per scene, reachable from anywhere through a static field. Every one of them uses the identical idiom:

```
public static GameManager Instance;    // the global access point

private void Awake()
{
    if (Instance != null && Instance != this)
    {
        Destroy(gameObject);           // a duplicate snuck into the scene – kill it
        return;
    }
    Instance = this;                   // claim the slot
}
```

Any script can then write `GameManager.Instance.GetState()` without needing a serialized reference or an expensive `FindObjectOfType` call.

Why singletons here?

This is a small, single-scene-at-a-time game. Each of these systems genuinely has exactly one of itself (one game state, one UI, one audio player, one crowd, one pool), and dozens of small scripts need to reach them. Serialized references would mean wiring the same manager into every gate, hurdle and enemy group in every scene — the static `Instance` is cheaper to build and impossible to forget to wire. The trade-off (hidden dependencies, harder unit-testing) is acceptable at this project size.

Singleton	File	Survives scene load?	Who uses it
<code>GameManager</code>	Managers/GameManager.cs	No	PlayerController, CrowdManager, AudioManager, HoldMoveUI, FinishLine
<code>LevelManager</code>	Managers/LevelManager.cs	Yes (DontDestroyOnLoad)	UIManager only
<code>UIManager</code>	Managers/UIManager.cs	No	GameManager
<code>AudioManager</code>	Managers/AudioManager.cs	No	GameManager, GateTrigger, EnemyGroup, SpinningHurdle, FinishLine
<code>CrowdManager</code>	Player/CrowdManager.cs	No	PlayerController, PlayerCharacter, GateTrigger, EnemyGroup, FinishLine
<code>ObjectPool</code>	Pool/ObjectPool.cs	No	CrowdManager, PlayerCharacter
<code>SplatPool</code>	Pool/SplatPool.cs	No	EnemyGroup, SpinningHurdle

Why is only LevelManager persistent?

`LevelManager` must remember which level you are on *across* scene loads, so it calls `DontDestroyOnLoad`. Every other manager describes the *current* scene, so it should die and be rebuilt with each level. Each level scene still contains a `LevelManager` object — the duplicate-check in `Awake` destroys the new copy when a persistent one already exists. It also re-syncs `currentLevelIndex` from the active scene's name, so pressing Play directly in Level2 still shows "LEVEL 2".

Singleton caveat

`Instance` is never set back to `null` in `OnDestroy`. It works because each new scene's manager overwrites the static field in `Awake` — but code running in the one frame around a scene load can see a destroyed instance. Don't cache `.Instance` across scene loads.

2.2 The state machine (GameManager)

The entire game phase lives in one enum on `GameManager`:

```
public enum GameState { Idle, Playing, Win, Lose }
```

- **Idle** — waiting for first touch. Movement disabled, tutorial visible.
- **Playing** — the run. Set by `StartPlaying()` on first touch.
- **Win / Lose** — terminal. Set exactly once by `Win()` / `Lose()`; both guard with `if (state != GameState.Playing) return;` so they can't fire twice or override each other.

How it's consumed: nothing subscribes to events — scripts simply *poll* `GameManager.Instance.GetState()` in `Update`. `PlayerController` early-returns unless Playing; `HoldMoveUI` hides itself the moment the state leaves Idle; `AudioManager` only plays footsteps while Playing.

Why polling instead of events?

With four states and a handful of consumers, polling is simpler to read and debug than an event system, and the per-frame cost of an enum comparison is zero. If the game grew more states or UI screens, switching to C# events/UnityEvents would be the natural upgrade.

2.3 Object pooling (ObjectPool & SplatPool)

Two systems create many objects at runtime: crowd characters (a ×3 gate can spawn 100+ in one frame) and death splats (a big battle stamps 70+ decals in seconds). `Instantiate` / `Destroy` at that rate causes garbage-collection hitches — very visible on Android. So both are **pooled**:

- `ObjectPool` pre-instantiates **300** inactive `PlayerCharacter`s in `Start`, parented under itself. `Get()` dequeues and activates one; `Return()` deactivates and re-enqueues it. If the pool runs dry it silently grows with `Instantiate`.
- `SplatPool` pre-instantiates **200** splat sprites. Splats are spawned on battle/hurdle deaths and **never despawn** during a level — the pool exists purely to avoid `Instantiate` spikes mid-battle, not to recycle.

The pool contract

Keep `ObjectPool.prewarmCount` (300) $\geq$ `CrowdManager.maxCount` (300). If `maxCount` ever exceeds the prewarm, the game still works but starts allocating mid-run — exactly the hitch pooling exists to prevent.

Enemies are NOT pooled

`EnemyCharacter.Die()` simply calls `Destroy()`. Enemies are hand-placed per level in modest numbers and never respawn, so pooling them would add complexity for no benefit. This asymmetry is intentional.

2.4 Composition over inheritance — and deliberate duplication

There are **no base classes and no interfaces** in this codebase. Behaviour is combined by putting several small MonoBehaviours on one `GameObject` and wiring them with serialized fields:

- The **Crowd Root** object carries `PlayerController` (input + movement) + `CrowdManager` (list + math) + `CrowdFormation` (packing). Each does one job.
- An **enemy group** carries `EnemyGroup` (battle logic) + `EnemyFormation` (packing), with `EnemyCharacter` children.

Why is `EnemyFormation` a copy-paste of `CrowdFormation`?

They are line-for-line near-identical, one operating on `List<PlayerCharacter>`, the other on `List<EnemyCharacter>`. This is a **deliberate project rule** inherited from the build spec: keep scripts flat and independently readable rather than introducing generics or a shared base class. **Do not "fix" this duplication.** If you change formation behaviour, change *both* files.

The hidden same-GameObject contract

`EnemyGroup` and `FinishLine` pause the player via `CrowdManager.Instance.GetComponent<PlayerController>()` — a component lookup, not a serialized reference. This silently requires `PlayerController` and `CrowdManager` to live on the **same GameObject** (they do: Crowd Root). If you ever split them apart, battles and the finish line will stop pausing the player with no error pointing at why.

2.5 The trigger & tag contract

Two different collision philosophies coexist, and knowing which is which saves hours:

Detector	What it looks for	Meaning
<code>GateTrigger</code> , <code>EnemyGroup</code> , <code>FinishLine</code>	<code>other.CompareTag("Player")</code>	Fires when the crowd as a whole arrives. Only one object in the scene has the Player tag: Crowd Root (SphereCollider, radius 0.25, isTrigger).
<code>SpinningHurdle</code>	<code>other.GetComponent<PlayerCharacter>()</code>	Fires per individual character — a hurdle should kill only the runners it physically touches, not the whole crowd.

Rule of thumb when adding content

"Does the whole crowd react once?" → tag check on the root. "Does each runner react individually?" → component check on the character.

2.6 Other patterns in one paragraph each

- **Shared-latch gates:** a two-choice gate is two `GateTrigger` s under one parent holding a `GateTriggerParent` with a single `HasTriggered` flag. Both children check the flag before firing, so a wide crowd can never consume both options. Reuse `Double Gate Trigger.prefab` for any multi-choice gate.
- **Inspector-first tuning:** every tunable is a `[SerializeField]` private field, set per-instance in the Inspector — never a constant in code. This lets designers tune per level, but read the serialization warning in section 7.5.
- **Choreography flag:** `CrowdManager.SetInBattle(true)` really means "*someone else owns the characters now*" — it suspends the disc formation so battles and the finish-line run-off can move characters directly. Setting it back to false makes survivors regroup automatically, with zero regroup code.
- **Fire-and-forget audio:** everything plays through one AudioSource via `PlayOneShot` , so sounds layer instead of cutting each other off, and there is exactly one place to control SFX.

3. Script reference — every component we wrote

Grouped by folder under `Assets/_Game/Scripts/`. Parameter tables for all of these are in section 5.

Two leftover scripts from a cut feature (`Boss/BossController.cs` , `UI/BossHealthBarUI.cs`) are unused by the shipped game and not covered here — ignore them.

3.1 Managers/

GameManager SINGLETON

The level's state machine (section 2.2). Holds `GameState { Idle, Playing, Win, Lose }`. `StartPlaying()` is called by `PlayerController` on the first touch and hides the tutorial. `Win()` (called by `FinishLine`) plays the win siren and shows the win panel; `Lose()` (called by `CrowdManager` when the crowd count hits zero) shows the lose panel. Both latch — once Win or Lose, the state never changes again until the scene reloads.

LevelManager SINGLETON · PERSISTENT

The only object that survives scene loads. Owns `levelSceneNames = ["Level1", "Level2"]` and the current index. `LoadNextLevel()` increments and **wraps back to Level1** after Level2; `ReloadLevel()` restarts the current scene. `GetCurrentLevelNumber()` feeds the "LEVEL n" label. In `Awake` it syncs its index to the actually-loaded scene, so Editor play-from-any-scene works.

UIManager SINGLETON

Owns references to the panels (hold-move tutorial, win, lose, plus a hidden legacy UI root) and the level label. `Start` sets initial visibility (tutorial on, everything else off) and writes the label. Exposes `ShowWinPanel()` , `ShowLosePanel()` (which shows the panel after a hardcoded 2-second `Invoke`), and the two button handlers `OnNextLevelButton` / `OnRetryButton` , wired to the scene buttons via UnityEvents (in the scene file, not in code).

AudioManager SINGLETON

One-stop SFX player. `Play(SfxType)` looks up the matching `SfxEntry` (type + clip variations + volume), picks a random clip and `PlayOneShot` s it on the single shared AudioSource. Its own `Update` drives the run-footstep loop: while the state is Playing it fires `SfxType.Run` every `runClipInterval` seconds — no other script needs to think about footsteps. The `SfxType` enum and hook table are in section 5.10.

3.2 Player/

PlayerController

On the **Crowd Root** (tagged Player). Reads mouse/touch: the first press while Idle starts the game; while Playing it moves the root forward at `forwardSpeed` every frame and maps horizontal finger drag to world X, clamped to `±laneHalfWidth` . The camera is a **child** of Crowd Root, so it follows for free — and freezing this script (via `SetPaused` , used by battles and the finish line) also freezes the camera. Movement is disabled on Win/Lose.

CrowdManager SINGLETON

The crowd's brain. Owns the `List<PlayerCharacter>` and is the **only** place crowd math happens: `AddCharacters` (pulls from the pool, parents under the root with a tiny random jitter so no two spawn coincident), `RemoveCharacters` (kills from the end of the list), `MultiplyCrowd`, `DivideCrowd`. Each change updates the count bubble. `UnregisterCharacter` is called by every dying character — when the list empties, it calls `GameManager.Lose()`. **This is the only lose trigger in the game.** `SetInBattle(bool)` gates the formation (section 2.6). Spawns `startCount` (1) in `Start`, caps at `maxCount` (300).

CrowdFormation

Packs the crowd into an organic disc every frame — no fixed slots, so gaps close naturally after deaths. Four steps per frame: (1) shift *all* characters uniformly so the blob's centroid sits on the root (a pure translation can't deform the shape); (2) pull inward **only** characters outside the computed packed-disc radius (a settled disc has nobody outside → perfectly still, no flicker); (3) `separationPasses` iterations of hard pairwise separation — any pair closer than `separationDistance` is pushed apart symmetrically, capped by `maxMoveSpeed`; (4) clamp world X to `±platformHalfWidth`. This is the **third** redesign of this system — see ProjectHistory.md Sessions 1–3 before changing it.

PlayerCharacter POOLED

One blue runner. Just an Animator wrapper: `SetRunning(bool)` flips the `"IsRunning"` bool. `Die()` is instant and synchronous: unregister from CrowdManager → stop the run anim → `ObjectPool.Return(this)`. No death animation, no delay — the count you see is always exact.

3.3 Enemy/

EnemyGroup

A stationary red crowd. `Start` collects every child `EnemyCharacter` (including inactive) and seeds its optional count bubble. When the Crowd Root's trigger enters, `BattleRoutine` runs the whole fight as one coroutine: pause the player (movement + camera), suspend the crowd's formation, then per frame — both sides charge their **nearest living opponent** at `chargeSpeed` while keeping `allySeparation` from teammates; any player/enemy pair within `killRange` trades kills **exactly 1-for-1** (a per-frame "claimed" set stops two enemies consuming the same player). Each trade stamps one blue + one red splat; one death sound plays per frame of trades (per *pair* would clip). When a side is wiped: bubble hidden, survivors regroup (or idle, if the enemies won), player unpaused only if the crowd survived — a wipe already routed to Lose via CrowdManager.

EnemyCharacter

Mirror of PlayerCharacter (same `"IsRunning"` animator contract) but **not pooled** — `Die()` is `Destroy(gameObject)`. Same model as the player, red material.

EnemyFormation

Line-for-line copy of CrowdFormation operating on enemies, so waiting groups idle in the same organic disc. Deliberate duplication — change both files together (section 2.4).

3.4 Level/

GateTrigger

One gate panel. Its operation is two Inspector fields: `op` (`Add / Subtract / Multiply / Divide`) and `value` . `Start` writes its own 3D label (" +25", "x2"...) from those fields — never type label text by hand. On crowd contact (tag check + its own `consumed` flag + the parent latch) it plays the gate sound and routes to the matching CrowdManager method. Always used in pairs via `Double Gate Trigger.prefab` .

GateTriggerParent

Three lines: `public bool HasTriggered { get; set; }` on the shared parent of a gate pair. Both children check it before firing and set it when they fire — the "only one choice counts" rule lives here.

SpinningHurdle

A rotating spiked obstacle. `Update` spins it around world Y at `rotateSpeed` °/sec. Its trigger kills **individual** `PlayerCharacter` s on contact (component check, not tag) — death sound + blue splat + instant `Die()` . Two prefab variants: `SpinningHurdle` (double-arm, spans most of the lane — hard) and `SpinningHurdleSingle` (single arm, ~3.4 m reach — easy).

FinishLine

The checkered line. On crowd contact: freeze the root (and camera), suspend the formation, hide the count bubble, play fireworks, then `RunOffRoutine` pushes every character forward at `runOffSpeed` **forever** (an intentional infinite loop — they jog into the distance behind the win panel) and fires `GameManager.Win()` after `winPanelDelay` (2 s). Its `boss` field is a legacy hook from a cut feature — always leave it empty.

3.5 Pool/

ObjectPool SINGLETON

The PlayerCharacter pool (section 2.3). `Get()` / `Return(pc)` . Prewarms 300 in `Start` ; grows silently if exhausted. Returned characters keep their old local position — that's fine only because `AddCharacters` always overwrites it on the next `Get()` .

SplatPool SINGLETON

The death-decal pool. `SpawnPlayerSplat(pos)` / `SpawnEnemySplat(pos)` place a flat sprite with: a random tint lerp between a light/dark color pair (blue pair for players, red for enemies), a random Y-rotation, a tiny stacked height (64-step ladder so overlapping splats never z-fight), and a 0 → `targetScale` pop-in eased over `scaleDuration` . Splats never despawn during a level.

3.6 UI/

HoldMoveUI

The "Hold & Move" tutorial prompt. Self-contained: while the state is Idle it slides the hand icon left/right on a sine wave (`swingDistance` , `swingSpeed`); the moment the state leaves Idle it `SetActive(false)` s itself. This is a deliberate belt-and-braces double with `UIManager.HideHoldMove()` — either alone would hide the prompt.

CountBubbleUI

A world-space number bubble. `LateUpdate` floats it `heightOffset` above whatever Transform is in `crowdRoot` and billboards it to the camera. Used twice: the crowd's bubble (driven by CrowdManager on every count change) and one per enemy group (driven by EnemyGroup each battle frame). Reusable for anything — the "crowdRoot" field accepts any transform.

4. Game flow, start to end

This is the exact runtime sequence, with the responsible script named at every step.

4.1 Scene load & bootstrap

1. **Awake** (order-independent): all seven singletons claim their `Instance`. LevelManager additionally survives loads and syncs its level index to the loaded scene's name. HoldMoveUI caches the hand icon's rest position.
2. **Start**: ObjectPool prewarms 300 characters; SplatPool prewarms 200 splats; CrowdManager spawns `startCount` (1) character; UIManager shows the tutorial, hides the win/lose panels, writes "LEVEL n"; every EnemyGroup collects its children and seeds its bubble; every GateTrigger writes its own label; every GateTriggerParent resets its latch.
3. State is **Idle**: the crowd stands at the start, the tutorial hand swings, nothing moves.

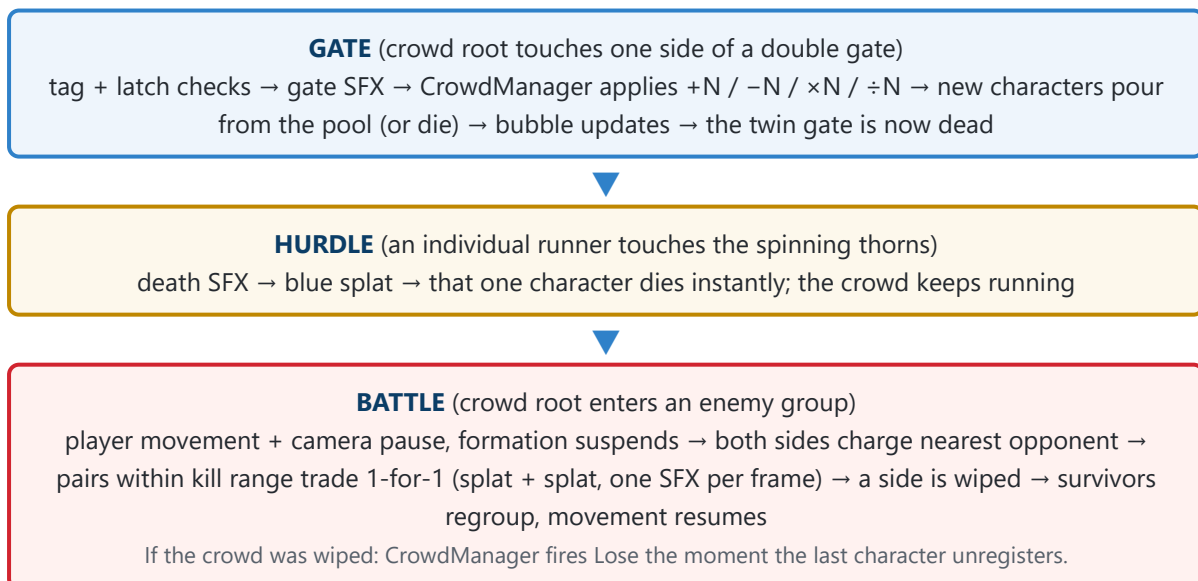
4.2 First touch

`PlayerController` sees the press while Idle → `GameManager.StartPlaying()` (state → Playing, tutorial hidden) and switches every character's animator to running. `AudioManager` begins the footstep loop.

4.3 The run

Each frame while Playing: PlayerController moves Crowd Root forward at `forwardSpeed` and maps drag to X (camera follows as a child) → CrowdManager asks CrowdFormation to re-pack the disc → CountBubbleUI floats and billboards the counter above the crowd.

4.4 The three obstacle types



4.5 Ending the level

WIN — FinishLine

root + camera freeze at the line → formation off,
bubble hidden → fireworks SFX → characters jog
forward forever → after 2 s `GameManager.Win()` →
win siren + Win panel

▼ *Next Level (UnityEvent → UIManager)*

`LevelManager.LoadNextLevel()` — Level1 →
Level2 → wraps to Level1

LOSE — CrowdManager

last character's `Die()` → `UnregisterCharacter`
sees count 0 → `GameManager.Lose()` → Lose panel
appears after a 2 s delay → everything freezes

▼ *Retry (UnityEvent → UIManager)*

`LevelManager.ReloadLevel()` — same level, fresh
state

The only lose path

There is exactly one way to lose: the crowd count reaching zero (battle, hurdle, or a –gate). Reaching the finish with 1 character is a win. There is no timer and no fall-off-the-edge condition.

5. Parameter reference & tuning guide

Every Inspector field of every component. **Code default** = the value written in the C# file; **scene value** = what is actually serialized in the levels today. Where they differ the scene value is shown in **bold red** — those four drifts are deliberate tuning experiments, but you must know they exist: changing the script default will **not** change the scene (section 7.5).

5.1 PlayerController — pace & steering

Field	Type	Code default	Scene value	What it tunes
<code>forwardSpeed</code>	float	6	5	Auto-run speed (units/s). The single biggest pace knob — raising it effectively shortens every reaction window in the level.
<code>steerSpeed</code>	float	12	12	Drag sensitivity: world X moved per full screen-width of finger travel. Too high feels twitchy, too low feels heavy.
<code>laneHalfWidth</code>	float	3	3	Hard clamp on crowd-root X. Matches the wall inner faces at ± 3 — only change together with the lane geometry.

5.2 CrowdManager — the crowd

Field	Type	Code default	Scene value	What it tunes
<code>formation</code>	CrowdFormation	—	same GameObject	Required reference. Not null-checked in Update — never unassign it (section 7.1).
<code>countBubble</code>	CountBubbleUI	—	assigned	The floating counter above the crowd.
<code>startCount</code>	int	1	1	Characters at level start. Raising it makes the opening easier and the first gate less dramatic.
<code>maxCount</code>	int	300	300	Hard cap on crowd size. Keep $\leq$ <code>ObjectPool.prewarmCount</code> or gates will Instantiate mid-run.

5.3 CrowdFormation / EnemyFormation — crowd shape

Identical fields on both scripts (deliberate copies). **These interact — read the tips below before touching.**

Field	Type	Code default	Scene value	What it tunes
<code>centerPullSpeed</code>	float	2.5	2.5	Cohesion drift (units/s) for characters outside the packed disc. Higher = stragglers rejoin faster.
<code>separationDistance</code>	float	0.55	player: 0.35 (L1 & L2) enemies: 0.55	The visual density knob. Minimum gap between any two characters; also feeds the packed-radius formula. The player crowd was deliberately packed tighter than enemies.
<code>separationPasses</code>	int	5	5	Relaxation iterations per frame. Measured: 2 passes settle at ~0.44 spacing, 5 hold ~0.53 of a 0.55 target. Cost is $O(\text{passes} \times n^2)$ — don't raise casually (section 7.4).
<code>maxMoveSpeed</code>	float	8	8	Caps separation push per second so a $\times 3$ gate's mass spawn resolves smoothly over a few frames instead of exploding.
<code>platformHalfWidth</code>	float	2.75	2.75	Per-character X clamp (lane floor is 6 wide, walls at ± 3 ; 0.25 margin keeps meshes off the walls).

Tuning the formation safely

Change `separationDistance` first and alone — it drives both look and the packed radius. If you see per-frame jitter after your change, you have re-created the flicker bug this system was redesigned three times to kill: read `ProjectHistory.md` Sessions 2–3 before going further. The packed-radius formula (`separationDistance × (0.59√n – 0.36)`) was calibrated ~8% above the measured rest radius — looser boundaries freeze oval shapes, tighter ones make the rim churn.

5.4 EnemyGroup — battles

Field	Type	Code default	Scene value	What it tunes
<code>chargeSpeed</code>	float	3.5	3.5	How fast both sides close (units/s). Higher = shorter, more chaotic fights.
<code>killRange</code>	float	0.6	0.6	Distance at which a player/enemy pair trades kills. The main fight-duration knob — larger resolves much faster.
<code>allySeparation</code>	float	0.45	0.45	Spacing kept from teammates while charging, so crowds meet as a front instead of a column.
<code>formation</code>	EnemyFormation	—	sibling component	Pre-battle disc packing.
<code>countBubble</code>	CountBubbleUI	—	assigned per group	Optional group-size bubble; auto-hides when the group is wiped.

Battles are always exactly 1-for-1

There is no damage, HP or randomness: N players entering a battle against M enemies always leaves $|N - M|$ survivors on the larger side. Difficulty is tuned purely by group sizes in the level, not here.

5.5 GateTrigger — the math gates

Field	Type	Default	What it tunes
<code>_parent</code>	GateTriggerParent	wired inside Double Gate Trigger.prefab	The shared one-choice latch. Required — a gate with this unassigned throws on contact (section 7.1).
<code>textLabel</code>	TextMeshPro	assigned in prefab	The 3D label. Text is auto-generated from op+value in Start — never edit the text itself.
<code>op</code>	Operator	Add	<code>Add / Subtract / Multiply / Divide</code> . Add/Subtract round <code>value</code> to int; Multiply/Divide use it as a float (×1.5 works).
<code>value</code>	float	25	The operand. Set per gate instance — see section 6 for every live gate's values.

5.6 SpinningHurdle

Field	Type	Code default	Prefab value	What it tunes
<code>rotateSpeed</code>	float	120	120 (both variants)	Degrees/sec around world Y. Negative reverses direction. Difficulty comes from speed × arm length: the double-arm prefab is the hard one, the single-arm (~3.4 m reach) the easy one.

5.7 FinishLine

Field	Type	Code default	Scene value	What it tunes
<code>boss</code>	BossController	null	null everywhere	Legacy hook from a cut feature — always leave it None.
<code>runOffSpeed</code>	float	6	(code default applies)	Victory jog speed past the camera.
<code>winPanelDelay</code>	float	2	(code default applies)	Seconds of victory jog before the Win panel. Note the lose side has a <i>separate, hardcoded 2 s</i> in UIManager (section 7.2).

5.8 ObjectPool & SplatPool

Component	Field	Type	Code default	Scene value	What it tunes
ObjectPool	<code>characterPrefab</code>	PlayerCharacter	—	PlayerCharacter.prefab	The pooled prefab.
ObjectPool	<code>prewarmCount</code>	int	300	300	Instances built at Start. Startup hitch vs. mid-run allocation trade-off. Keep $\geq$ CrowdManager.maxCount.
SplatPool	<code>splatPrefab</code>	SpriteRenderer	—	Splat.prefab	The flat decal sprite (must be authored upright; spawn lays it flat with a 90° X rotation).
SplatPool	<code>prewarmCount</code>	int	100	200	Splats built at Start — sized to the biggest battle (largest group $\times$ 2 splats per trade).
SplatPool	<code>playerLight / playerDark</code>	Color	light/dark blue pair	same	Each player splat gets a random tint between the pair — variety for free.
SplatPool	<code>enemyLight / enemyDark</code>	Color	light/dark red pair	same	Same for enemy splats.
SplatPool	<code>targetScale</code>	float	0.2	0.2	Final decal size.
SplatPool	<code>scaleDuration</code>	float	0.25	0.1	Pop-in time (cubic ease-out). Was shortened — at 0.25 the pop lagged the kill beat.

5.9 UI components

Component	Field	Type	Default / scene	What it tunes
UIManager	<code>holdMovePanel</code> / <code>winPanel</code> / <code>losePanel</code> / <code>bossHealthBarRoot</code>	GameObject	assigned per scene	Panel references. <code>losePanel</code> is dereferenced without a null check (section 7.1). <code>bossHealthBarRoot</code> is a legacy reference from a cut feature — it stays hidden forever.
UIManager	<code>levelText</code>	TMP_Text	assigned	The "LEVEL n" label. It is a child of <code>HoldMoveUI</code> , so it hides on first touch — intentional layout.
HoldMoveUI	<code>hand</code>	RectTransform	wired per scene	The tutorial hand icon.
HoldMoveUI	<code>swingDistance</code>	float	150	Hand travel in canvas px to each side.
HoldMoveUI	<code>swingSpeed</code>	float	0.6	Full swing cycles per second.
CountBubbleUI	<code>crowdRoot</code>	Transform	set per instance	What the bubble follows — any transform works (the crowd uses <code>Crowd Root</code> ; each enemy bubble uses its group).
CountBubbleUI	<code>countText</code>	TMP_Text	assigned	The number label.
CountBubbleUI	<code>heightOffset</code>	float	2.2	World units above the followed transform.
GameManager	<code>state</code>	GameState	Idle	Debug readout of the current phase — leave it <code>Idle</code> in the scene.
LevelManager	<code>levelSceneNames</code>	string[]	["Level1", "Level2"]	Play order. Names must match Build Settings <i>exactly</i> (loaded by string).
LevelManager	<code>currentLevelIndex</code>	int	0 (auto-synced)	0-based current level; corrected in <code>Awake</code> from the active scene name.

5.10 AudioManager — SFX table

Field	Type	Default / scene	What it tunes
<code>sfxSource</code>	AudioSource	assigned (2D, <code>playOnAwake</code> off)	The single shared source. Falls back to <code>GetComponent</code> in <code>Awake</code> .
<code>effects</code>	SfxEntry[]	7 entries, one per SfxType	Each entry: <code>type</code> + <code>clips[]</code> (random variation pool) + <code>volume</code> [Range 0–1]. A missing entry means <i>silent, no warning</i> .
<code>runClipInterval</code>	float	0.35	Seconds between footstep one-shots while <code>Playing</code> . Lower = faster-sounding run — tune together with <code>forwardSpeed</code> .

Every `SfxType` and where it is triggered from:

SfxType	Clips	Played by
DeadPlayer	12 variations	EnemyGroup — once per <i>frame</i> of battle trades (per pair would clip); SpinningHurdle — per kill
DeadPlayerLast	1	Nothing — reserved/unhooked. Clip is assigned but no script calls it.
EnemyCrowdWin	1	EnemyGroup — when the player crowd is wiped
GateTrigger	1	GateTrigger — on gate consume
Run	8 variations	AudioManager itself — footstep loop while Playing
WinFireworks	1	FinishLine — the moment the line is crossed (layers under the siren)
WinSiren	1	GameManager.Win()

6. Level layouts & balance

Both levels follow the same rhythm: one "beat" (gate, enemy group or hurdle) every fixed z-distance, so the player always has recovery time between decisions. The count bubble is the player's scoreboard — the wide spacing exists so they can absorb each change before the next beat.

6.1 Level 1 — the teaching level (finish z = 85, beat every 15)

z	Beat	Left choice	Right choice / size
10	Double gate	+25	+50
25	Enemy group	15 enemies (own count bubble)	
40	Double gate	×2	+30
55	Enemy group	20 enemies (own count bubble)	
70	Double gate	×3	+40
85	Finish line		

No hurdles in Level 1 — it teaches gates and battles only. Balance: even the worst-choice path survives the second group with 2 characters; a good natural run reaches the finish with ~92–138.

6.2 Level 2 — the real test (finish z = 106, beat every 12)

z	Beat	Left choice	Right choice / size
10	Double gate	+60	+100
22	Hurdle	SpinningHurdle (double-arm — spans most of the lane)	
34	Enemy group	30 enemies	
46	Double gate	×2	+60
58	Enemy group	50 enemies	
70	Hurdle	SpinningHurdleSingle (single arm, ~3.4 m reach)	
82	Double gate	×3	+80
94	Enemy group	35 enemies	
106	Finish line		

(Exact beat positions follow the every-12 rhythm; Level2's lane floor and walls were extended to z = −10...120 to fit.)

Balance numbers assume zero hurdle deaths

The gate/group arithmetic was verified in automated runs **with hurdles disabled**, because touch steering can't be simulated through our tooling. Real difficulty with hurdles active has only been sanity-checked by hand. If a level feels unfair, tune group sizes first (cheapest lever), gate values second.

Adding a beat to a level

Duplicate an existing **Double Gate Trigger** instance or enemy group in the scene (they carry all correct wiring, including the latch and the count bubble), move it to a free z slot on the rhythm, and set **op** / **value** or add/remove enemy children. Both scenes also have a **Buildings** root of decorative collider-less cubes flanking the lane — scenery only.

7. Known issues & gotchas — the survival guide

7.1 Unchecked references (these WILL throw if violated)

Where	What must be true	What happens otherwise
<code>GateTrigger._parent</code>	Every gate must have its parent latch wired (the Double Gate Trigger prefab does this)	A bare <code>GateTrigger.prefab</code> instance throws <code>NullReferenceException</code> the moment the crowd touches it — the prefab ships with <code>_parent</code> empty. Always place the Double prefab.
<code>CrowdManager.formation</code>	Must stay assigned	Exception every frame from Update.
<code>SpinningHurdle</code> → <code>SplatPool.Instance</code>	A <code>SplatPool</code> must exist in the scene	Throws on the first hurdle kill (<code>EnemyGroup</code> null-checks the pool; the hurdle doesn't).
<code>UIManager.losePanel</code>	Must stay assigned	Throws 2 s after losing (inside the delayed <code>LostPanel</code> Invoke).
<code>HoldMoveUI</code> → <code>GameManager.Instance</code>	<code>GameManager</code> must Awake before <code>HoldMoveUI</code> updates	Throws on the first frame if the <code>GameManager</code> object is missing/disabled.

7.2 Magic numbers & twin delays

- **Two independent 2-second delays** gate the end panels: the win side uses the field `FinishLine.winPanelDelay` (tunable), the lose side uses a **hardcoded** `Invoke("LostPanel", 2f)` inside `UIManager` (not tunable from the Inspector). If you retune one, remember the other.
- The animator bool is the string `"IsRunning"` in both character scripts — renaming the parameter in the `AnimatorController` silently breaks running animations.
- Scene loading is **by scene-name string** — renaming a scene requires updating `LevelManager.levelSceneNames` and Build Settings together.
- Formation internals (re-centre deadband 0.05, packed-radius formula, coincident-pair epsilon) are commented in the source with the measurements that produced them — treat them as calibrated constants.

7.3 Stale & dead things (don't be fooled)

- `EnemyGroup.prefab` **is stale/broken** — its serialized data contains only a field that no longer exists (`hitInterval`). All three live enemy groups are plain scene objects. To add a group, duplicate one *in the scene*, don't drag this prefab.
- `FinishLine.prefab` still carries a dead `staircase` field from the removed pyramid ending — harmless, ignore it.
- Dead APIs with zero callers, kept intentionally: `LevelManager.LoadLevel(int)` (for a future menu), `CrowdManager.RemoveFromTracking` (leftover from the removed staircase), `SfxType.DeadPlayerLast` (reserved).
- Leftovers from a cut feature — ignore them all: `Boss/BossController.cs`, `UI/BossHealthBarUI.cs`, `BossCharacter.prefab`, the hidden `BossHealthBar` instance referenced by `UIManager.bossHealthBarRoot`, and `FinishLine`'s empty `boss` field.

- `CrowdManager.RefreshAfterChange()` only updates the count bubble, despite its comment claiming more.

7.4 Performance notes (fine today, watch when scaling)

- Both formation scripts are **$O(\text{separationPasses} \times n^2)$** per frame. At the 300-character cap that is $5 \times 90,000$ pair checks — acceptable, but don't raise the cap or the passes without profiling on a device.
- `CrowdManager.GetCharacters()` returns a **new List copy on every call** and is called every frame during battles and the finish run-off. Safe, but a known allocation source.
- `CountBubbleUI` calls `Camera.main` every LateUpdate (an internal find-by-tag). Cache it if bubbles ever multiply.
- A wide hurdle sweep through a dense crowd stacks one `DeadPlayer` one-shot per kill in the same frame — audibly loud, not a bug.

7.5 The golden serialization rule

Changing a script default does NOT change existing scenes

Once a field has been serialized into a scene or prefab, it keeps that value forever — editing the C# default only affects *newly added* components (and brand-new fields, which do pick up the code default everywhere). This is why `forwardSpeed` is 5 in the scenes but 6 in code, and why several past "bugs" were really stale scene values.

Corollaries:

- To retune a live level, edit the **scene component**, not the script.
- When live behaviour contradicts what the code says, diff the Inspector values against the code defaults *first*.
- Unsaved Inspector experiments persist for the whole Editor session and carry into Play mode — two past debugging sessions were burned on this.